

MEDIATEK

MT8676 Android Sensor Issue Debug SOP

版本: 1.0
出版日期: 2025-07-11

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2025-07-11	张厚松	正式版

目录

版本记录	2
目录	3
表格目录	4
1 概述	5
1.1 目的	5
2 缩略词	6
2.1 缩略词	6
3 VTS 测试失败	7
4 APK 无法获取 Sensor 数据	8
4.1 SCP 层 Sensor 异常 Debug	8
4.1.1 检查 SCP 是否正常启动	8
4.1.2 检查 Sensor Device 是否初始化成功	8
4.1.3 DVFS 导致 Sensor 异常	9
4.2 Kernel 层 Sensor 异常 Debug	10
4.2.1 Dump Sensor List	10
4.2.2 Kernel 端异常排查	11
4.2.3 系统 Suspend 之后, Resume 时出现 I2C Error	12
4.3 HAL 层 Sensor 异常排查	13
4.3.1 HAL 层文件检查	13
4.3.2 Sensor HAL 上报数据频率异常	13
5 Sensor 校准失败	14
6 Sensor Debug Tools 介绍	15
6.1 Sensor Debug APK	15
6.2 测试工具 high_freq_sensor_tool	17
6.2.1 基于 high_freq_sensor_tool 测试 Sensor 是否正常	17
6.3 基于 high_freq_sensor_tool 做 Sensor 校准	18
6.3.1 ACC 校准方法	18
6.3.2 GYRO 校准方法 (与 ACC 相同)	20
6.3.3 PS 校准方法	20
7 如何抓取 Sensor Log	21
8 案例分析	25

8.1 ALPS08300249 平板 8 字校准后，地磁数据与机械指南针对比相差大于 5 25

附件一 附加条款 26

表格目录

表 2-1. 缩略词 6

1 概述

1.1 目的

本文档主要介绍 MT8676 Sensorhub 相关问题的调试方法，包括 SCP，Kernel 和 Sensor HAL 三部分在出现问题时的 debug 手段，以及在 Sensor 校准时的相关注意事项。

由于 Sensor 涉及到 HAL 层，Kernel 层以及 SCP 层，当 APK 获取 Sensor 数据失败时，通常需要从以上三个方向查找原因。

结合以往的 debug 经验：SCP 层出错的概率高于 HAL 层，HAL 层出错的概率高于 Kernel 层。

2 缩略词

2.1 缩略词

表 2-1. 缩略词

缩略词	解释
MSBPD	MediaTek Sensor Boost Path Driver; a driver that runs in Linux using MSBP API
MSBPKI	MediaTek Sensor Boost Path Kernel space Interface; support API for kernel space
MSHBP	MediaTek Sensor Hbu Broadcast Path; the architecture implemented in Sensor Hub we execute applications or drivers
MSHBPA	MediaTek Sensor Hbu Broadcast Path; an application or driver that runs within MSHBP using MSHBP API
MUSA	MediaTek Universal Sensor Architecture; the common concept including MSBP and MSHBP

3 VTS 测试失败

导出 VTS 测试 log

选择 host_log

device_compatibility_matrix.xml_1704...	2022/2/18 8:53	WinRAR archive
device_logcat_setup_0123456789ABC...	2022/2/18 8:53	WinRAR archive
device_logcat_tearndown_0123456789...	2022/2/18 8:54	WinRAR archive
device_logcat_test_0123456789ABCD...	2022/2/18 8:54	WinRAR archive
device_manifest.xml_13549210052715...	2022/2/18 8:53	WinRAR archive
executeShellCommandLog_01234567...	2022/2/18 8:54	WinRAR archive
framework_compatibility_matrix.xml_...	2022/2/18 8:53	WinRAR archive
framework_manifest.xml_6846968245...	2022/2/18 8:53	WinRAR archive
host_adb_log_1052267898012641387...	2022/2/18 8:54	WinRAR archive
host_log_1813023215301103302.txt.gz	2022/2/18 8:54	WinRAR archive

找出报错的 Sensor 类型

```
02-18 16:55:02 I/ModuleVibrator: [1/1] VtsHwSensorV2_ViaTargetTest: FellInstance/SensorHwTest/Activate/0 default FAILURE hardware/interfaces/sensors/common
Expected equality of these values:
activate(sensor.sensorHandle, true)
Which is: 40-byte object <00-00 00-00 00-00 00-00 00-00 00-00 00-00 00-00 00-00 00-00 00-00 00-00 00-00 00-00 00-00 00-00 01-00 00-00 FF-FF FF-FF>
Result::OK
Which is: OK
Google Test trace:
hardware/interfaces/sensors/common/vts/2_X/VtsHwSensorsV2_XTargetTest.h:714: handle=0x00000017 type=2 name=WAKE_GESTURE
```

确认当前 project 是否支持该 Sensor

不支持

支持

取消 VTS 对该 Sensor 的测试

在 ProjectConfig.mk 中将该 Sensor 配置为 no

例如:

```
CUSTOM_KERNEL_VIBRATOR =
CUSTOM_KERNEL_WAKE_GESTURE_SENSOR = no
CUSTOM_MODEM =
```

参考 APK 无法获取 Sensor 数据的解法

仍然无解

寻求 MTK 协助

如果您的问题是 VTS 测试失败, 请参考以上流程图先自行检查, 如果确实无法解决, 请寻求 MTK 协助, 并且附上您 VTS 测试结果, 以便我们协助分析。

4 APK 无法获取 Sensor 数据

4.1 SCP 层 Sensor 异常 Debug

4.1.1 检查 SCP 是否正常启动

Sensor 在 probe 阶段会通过 IPI 与 SCP 通信，如果 SCP 启动异常会有如下 log：

```
/* register ipi interrupt handler */
err = scp_ipi_registration(IPI_SENSOR, mtk_nanohub_ipi_handler,
    "mtk_nanohub");
if (err != SCP_IPI_DONE) {
    pr_err("IPI_SENSOR register fail, err:%d\n", err);
    goto exit_wakeup_source_unreg_2;
}
```

使用 `cat /sys/class/misc/scp/scp_A_get_last_log`

如果 SCP 启动正常，能看到完整的 SCP log

部分截图如下：

```
ve 0
[0.010](0) pin rcv table:
[0.010](0) mbox0 pin00 index: 00, offset: 00, sz: 09
[0.010](0) mbox1 pin03 index: 00, offset: 00, sz: 02
[0.010](0) mbox1 pin04 index: 01, offset: 02, sz: 01
[0.010](0) mbox1 pin05 index: 02, offset: 04, sz: 02
[0.010](0) mbox1 pin06 index: 03, offset: 06, sz: 01
[0.010](0) mbox1 pin26 index: 04, offset: 08, sz: 09
[0.010](0) mbox1 pin33 index: 09, offset: 18, sz: 10
[0.010](0) mbox2 pin11 index: 00, offset: 00, sz: 04
[0.010](0) mbox2 pin29 index: 02, offset: 04, sz: 16
[0.010](0) mbox2 pin31 index: 10, offset: 20, sz: 07
[0.010](0) pin send table:
[0.010](0) mbox0 pin01 index: 00, offset: 00, sz: 02
[0.010](0) mbox0 pin02 index: 00, offset: 00, sz: 26
[0.010](0) mbox1 pin07 index: 14, offset: 28, sz: 02
[0.010](0) mbox1 pin08 index: 15, offset: 30, sz: 10
[0.010](0) mbox1 pin09 index: 20, offset: 40, sz: 01
[0.010](0) mbox1 pin10 index: 21, offset: 42, sz: 02
[0.010](0) mbox1 pin27 index: 22, offset: 44, sz: 02
[0.010](0) mbox1 pin28 index: 23, offset: 46, sz: 05
[0.010](0) mbox1 pin05 index: 26, offset: 52, sz: 01
[0.010](0) mbox1 pin34 index: 27, offset: 54, sz: 10
[0.010](0) mbox2 pin12 index: 14, offset: 28, sz: 04
[0.010](0) mbox2 pin30 index: 16, offset: 32, sz: 02
[0.010](0) mbox2 pin32 index: 17, offset: 34, sz: 07
[0.010](0) Build time: Apr 9 2022 16:21:01
[0.010](0) ram dump: 0x27388
[0.012](1) [PWRAP]Read pass, ret=0
[0.012](1) [PWRAP]Write pass
[0.012](1) [PWRAP]init done
[0.012](1) RV33_1: dvfs_init done
```

4.1.2 检查 Sensor Device 是否初始化成功

1. 由于 SPI/I2C 通信引起的 Sensor 初始化失败

查看 `cat /sys/class/misc/scp/scp_A_get_last_log`

例如:

```
[6.316](0) DEPUTY.ltr308 task 1 id 0
[6.316](0) addr: 53, transfer ACK error
[6.316](0) addr: 53, transfer ACK error
[6.316](0) addr: 53, transfer ACK error
[6.316](0) DEPUTY.ltr308 probe fail -19
[6.316](0) DEPUTY.module init ltr308 fail on task 1
```

如果 log 中有出现 SPI/I2C error, 首先请确认:

device 是否有挂载, I2C/SPI channel 以及 GPIO mode 配置是否正确

I2C error 请参考: *I2C Customer Support SOP*

SPI error 请参考: *SPI Customer Support SOP*

2. 由于 IPI 通信导致的 error

如果 AP 或者 SCP log 中出现 IPI timeout 字样

检查 AP 和 SCP 端 IPI_SENSOR 注册的 ID 是否一致

AP: drivers/misc/mediatek/scp/include/scp.h

```
enum ipi_id {
    IPI_MPOOL,
    IPI_CHRE,
    IPI_CHREX,
    IPI_SENSOR,
    IPI_SENSOR_INIT_START,
    SCP_NR_IPI,
};
```

SCP: vendor/mediatek/proprietary/tinysys/scp/drivers/project/ipi_legacy_wrap.h

```
enum ipi_id {
    IPI_MPOOL,
    IPI_CHRE,
    IPI_CHREX,
    IPI_SENSOR,
    IPI_HDMIRX,
    IPI_HDMICEC,
    IPI_WRAPPER_TOTAL,
};
```

4.1.3 DVFS 导致 Sensor 异常

1. DVFS 导致 Sensor 休眠

现象:

从 log 看, Sensor 初始化正常

Sensor 获取数据失败且 SCP 无 log 印出

2. DVFS 降低 CPU 工作频率

现象:

Sensor 通信正常, 但获取数据速度比较慢 (约十几秒获取一包数据)

针对上述两种现象的解法:

先临时 Disable DVFS 功能后测试, 如果以上问题仍然存在, 请寻求 MTK 帮助

Disable DVFS 方法:

AP: drivers/misc/mediatek/scp/include/scp_feature_define.h

设置 SCP_DVFS_INIT_ENABLE (0)

```
#define SCP_DVFS_INIT_ENABLE (0)
```

SCP: vendor/mediatek/proprietary/tinysys/scp/project/platform/platform.mk

设置 CFG_VCORE_DVFS_SUPPORT = no

```
CFG_VCORE_DVFS_SUPPORT = no
```

注: SCP 和 AP enable/disable DVFS 需要保持同步

4.2 Kernel 层 Sensor 异常 Debug

4.2.1 Dump Sensor List

首先需要看一下 Sensor List, 来确认是否是 AP 侧 Kernel 中出现的异常:

理论上, 如果 SCP 端 Sensor 工作正常, Dump Sensor List 是可以看到当前有哪些 Sensor 的:

```
[6.385](0) DEPUTY.ltr308 task 1 id 0
[6.386](0) addr: 53, transfer ACK error
[6.386](0) addr: 53, transfer ACK error
[6.386](0) addr: 53, transfer ACK error
[6.386](0) DEPUTY.ltr308 probe fail -19
[6.386](0) DEPUTY.module init ltr308 fail on task 1
[6.386](0) PRINCIPAL.transceiver task 0 id 0
[6.387](0) PRINCIPAL.mc34x9task 0 id 1
[6.387](0) PRINCIPAL.mc34x9mc34x9 bus num: 2,eint 7
[6.387](0) PRINCIPAL.mc34x9mc34x9 detected and device id 0xa4
[6.387](0) PRINCIPAL.mc34x9mc34x9 detect ret:0
[6.388](0) PRINCIPAL.mc34x9mc34x9_init DONE
[6.388](0) PRINCIPAL.uncali task 0 id 2
[6.388](0) PRINCIPAL.uncali can't find sensor 2
[6.388](0) PRINCIPAL.uncali can't find sensor 3
[6.390](0) PRINCIPAL.mc34x9ready
```

例如，在如上 SCP log 中，光感 ltr308 工作异常，ACC 工作正常，那我们在 AP 端 Dump Sensor List 是可以看到 ACC Sensor 的。

```
opal:/ #
opal:/ # dumpsys sensorservice
Captured at: 10:05:56.426
Sensor Device:
Total 8 h/w sensors, 8 running 0 disabled clients:
Sensor List:
0x00000001) ACCELEROMETER | mc34x9_acc | ver: 1 | type: android.sensor.accelerometer(1) | perm: n/a | flags: 0x00000000
continuous | minRate=12.50Hz | maxRate=400.00Hz | FIFO (max,reserved) = (4500, 3000) events | non-wakeUp |
0x00000011) SIGNIFICANT_MOTION | MTK | ver: 1 | type: android.sensor.significant_motion(17) | perm: n/a | flags: 0x00000005
one-shot | maxDelay=0us | minDelay=1us | no batching | wakeUp |
0x00000016) TILT_DETECTOR | MTK | ver: 1 | type: android.sensor.tilt_detector(22) | perm: n/a | flags: 0x00000007
special-trigger | maxDelay=0us | minDelay=0us | no batching | wakeUp |
0x00000017) WAKE_GESTURE | MTK | ver: 1 | type: android.sensor.wake_gesture(23) | perm: n/a | flags: 0x00000005
one-shot | maxDelay=0us | minDelay=1us | no batching | wakeUp |
0x00000018) GLANCE_GESTURE | MTK | ver: 1 | type: android.sensor.glance_gesture(24) | perm: n/a | flags: 0x00000005
one-shot | maxDelay=0us | minDelay=1us | no batching | wakeUp |
0x0000001b) DEVICE_ORIENTATION | MTK | ver: 1 | type: android.sensor.device_orientation(27) | perm: n/a | flags: 0x00000002
on-change | minRate=1.00Hz | minDelay=0us | no batching | non-wakeUp |
0x0000001d) STATIONARY_DETECT | MTK | ver: 1 | type: android.sensor.stationary_detect(29) | perm: n/a | flags: 0x00000005
one-shot | maxDelay=0us | minDelay=1us | no batching | wakeUp |
0x0000001e) MOTION_DETECT | MTK | ver: 1 | type: android.sensor.motion_detect(30) | perm: n/a | flags: 0x00000005
one-shot | maxDelay=0us | minDelay=1us | no batching | wakeUp |
Fusion States:
9-axis fusion disabled (0 clients), gyro-rate= 200.00Hz, q=< 0, 0, 0 > (0), b=< 0, 0, 0 >
game fusion(no mag) disabled (0 clients), gyro-rate= 200.00Hz, q=< 0, 0, 0 > (0), b=< 0, 0, 0 >
geomag fusion (no gyro) disabled (0 clients), gyro-rate= 200.00Hz, q=< 0, 0, 0 > (0), b=< 0, 0, 0 >
Recent Sensor events:
Active sensors:
Socket Buffer size = 984 events
Wakelock Status: not held
Mode : NORMAL
Sensor Privacy: disabled
0 active connections
0 direct connections
Previous Registrations:
opal:/ #
```

如果 SCP 端 Sensor 工作正常，在 AP 端 Dump Sensor List 却没有看到该 Sensor，那 AP 端 Sensor Flow 可能会有异常。

4.2.2 Kernel 端异常排查

Kernel 端 Sensor Flow 一般不会改动，出错的概率比较低，一般 Kernel 端出错，大多数都是由于配置错误引起，主要有两个方面：

1. 检查 Config 文件中 Sensor 架构是否正确

目前常用的架构有 1.0 和 2.0，需要和 device 目录下 ProjectConfig.mk 保持一致

```

200 CONFIG_MTK_SENSOR_SUPPORT=y
201 CONFIG_MTK_SENSOR_ARCHITECTURE="1.0"
202 CONFIG_MTK_SLBC_MMSRAM=y

```

2. 检查 Config 文件中 Sensor 配置是否与料搭配

例如，光感的料有 ltr308/ltr559/ltr578 等多种：

```

obj-$(CONFIG_MTK_LTR559) += ltr559/
obj-$(CONFIG_MTK_LTR578) += ltr578/

```

务必要根据 Sensor 的选料配置好 Config 文件：

```

204 CONFIG_MTK_MC34XX=y
205 CONFIG_CUSTOM_KERNEL_ALSPS=y
206 CONFIG_MTK_LTR559=y
207 CONFIG_MTK_HWMON=y

```

4.2.3 系统 Suspend 之后，Resume 时出现 I2C Error

部分供应商的 vendor driver 加入了 suspend/resume 接口。在 suspend 时会 disable driver 以达到省电目的。

```

static int stk3x3x_i2c_suspend(struct device *dev)
{
    int err;
    struct i2c_client *client = to_i2c_client(dev);
    struct stk3x3x_priv *obj = i2c_get_clientdata(client);

    APS_LOG("stk3x3x_i2c_suspend\n");

    if (!obj) {
        APS_ERR("null pointer!!\n");
        return 0;
    }

    atomic_set(&obj->als_suspend, 1);
    err = stk3x3x_enable_als(obj->client, 0);
    if (err)
        APS_ERR("disable als fail: %d\n", err);

    return 0;
}

```

但该逻辑和上层代码有一定冲突，resume 时由于上层的 polling 定时器未关，可能会造成在器件驱动的 resume 执行到之前就触发了 polling data 从而引起异常。

解决方法是将 suspend/resume 的接口移动到上层代码中，避免出现不同步的问题。

4.3 HAL 层 Sensor 异常排查

4.3.1 HAL 层文件检查

Sensor HAL 的正常工作依赖于平台里的以下文件：

- **HIDL Interface:**
`/system/lib64/android.hardware.sensors@1.0.so`
`/system/lib64/android.hardware.sensors@2.0.so`
`/system/lib64/android.hardware.sensors@2.1.so`
- **Multi-HAL**
`/vendor/bin/hw/android.hardware.sensors@2.0-serivce-mediatek`
`/vendor/etc/init/android.hardware.sensors@2.0-serivce-mediatek.rc`
`/vendor/lib64/android.hardware.sensors@2.X-subhal-mediatek.so`
- **HAL**
`/vendor/lib64/hw/sensors.$(TARGET_BOARD_PLATFORM).so`
`/vendor/etc/init/hw/init_sensor_2_0.rc`
- **Hfmanager**
`/vendor/lib64/libhfmanager.so`

如果 AP 端 Dump Sensor List 正常，可以排查以上文件是否存在来确认 HAL 层是否出现异常。如果发现有文件缺失，请查看 ProjectConfig.mk 中 MTK_SENSOR_SUPPORT 是否打开，若确认打开却仍然缺少文件，请寻求 MTK 协助。

4.3.2 Sensor HAL 上报数据频率异常

首先检查 SCP log，查看 transceiver 是否打出正确的 delay 和 latency

```
[336.816](0)[0] deputy.motion_acc_magnitude: 0.203333  
[336.816](0)[0] principal.transceiver enable: 1 20000000 0
```

如果能打出正确的 delay 和 latency，请检查对应 Sensor Driver 中的 odr 值，有些 Sensor 的频率是根据上层下来的取样间隔与自身支持的 odr 结合判断频率的。

如果 delay 和 latency 不正常，请寻求 MTK 协助。

如果是多 OS 系统，请在 Host OS 上使用工具测试，如果在 Host OS 上测试正常，但是在 Guest OS 上不正常，说明是虚拟化问题，请寻找 MTK 协助。如果在 Host OS 上也不正常，则按照前面的方法排查。

5 Sensor 校准失败

Sensor 在执行校准的时候，如果最后的返回值不是 0，则表示校准失败。

注：校准方法请参考本文 6.1 节。

```

C:\Windows\System32>adb shell
k6885v1_64:/ # ./vendor/high_freq_sensor_tool -c 1,3,0,0
mainThread <4953,4953>
command: [1,3,0,0]
looperThread <4953,4954>
data: [1,3,4495353781,-213.000000,-85.000000,-123.000000,0]
^Csignal_handler0=2 <4953,4953>
signal_handler1=3 <4953,4954>
mainThread exit
./vendor/high_freq_sensor_tool -c 1,3,0,0
mainThread <4955,4955>
command: [1,3,0,0]
looperThread <4955,4956>
data: [1,3,4495353654,-213.000000,-85.000000,-123.000000,1]
^Csignal_handler0=2 <4955,4955>
signal_handler1=3 <4955,4956>
mainThread exit
k6885v1_64:/ #
  
```

0,success

Not 0,

校准失败主要都是因为精度不足或者零漂过大引起。

常用的解法有以下两种：

1. 咨询 vendor，调整 Sensor 采集的精度
2. 增大 Sensor 校准的阈值

```

const struct GyroCalParameters gyro_cal_parameters = {
    .min_still_duration_nanos = SEC_TO_NANOS(5.9f),
    .max_still_duration_nanos = SEC_TO_NANOS(6.9f),
    .calibration_time_nanos = 0,
    .window_time_duration_nanos = SEC_TO_NANOS(1.5f),
    .bias_x = 0,
    .bias_y = 0,
    .bias_z = 0,
    .stillness_threshold = 0.95f,
    .stillness_mean_delta_limit = MDEG_TO_RAD * 40.0f,
    .gyro_var_threshold = 5e-5f,
    .gyro_confidence_delta = 1e-5f,
    .accel_var_threshold = 8e-3f,
    .accel_confidence_delta = 1.6e-3f,
    .mag_var_threshold = 1.4f,
    .mag_confidence_delta = 0.25f,
    .temperature_delta_limit_celsius = 1.5f,
    .gyro_calibration_enable = true,
};
  
```

6 Sensor Debug Tools 介绍

6.1 Sensor Debug APK

环境需求：Android 平台

安装方法：adb install GameCube_v3.1.apk

```
C:\Users\mtk20889>adb install C:\Users\mtk20889\Desktop\ARDC(B1332)\GameCube_v3.1.apk
Performing Streamed Install
Success
```

安装成功后，可以在屏幕看到如下图标：



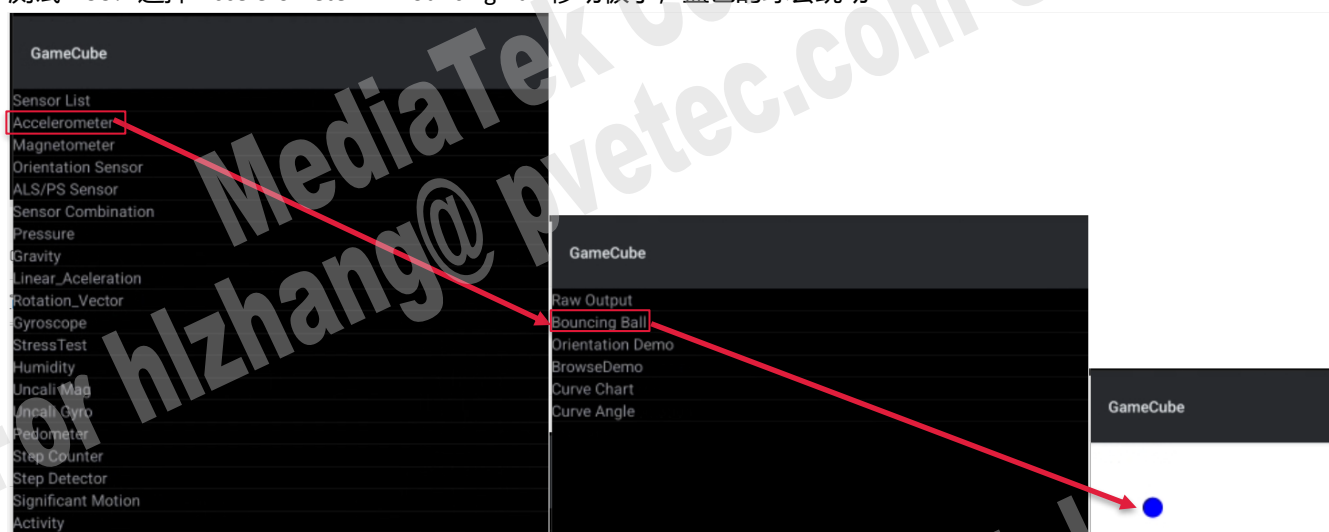
debug 方法：

1. 打开通过 adb 安装好的 APK，界面如下：



2. 以 ACC 和 ALPS 为例

测试 ACC：选择 Accelerometer → Bouncing Ball 移动板子，蓝色的球会跳动



测试 ALS/PS：选择 ALS/PS → Curve Chart 左上角会有光感的数值，遮挡光感数值会变小



6.2 测试工具 high_freq_sensor_tool

环境需求：Android 平台/Yocto 平台

6.2.1 基于 high_freq_sensor_tool 测试 Sensor 是否正常

1. 如果平台没有自带 high_freq_sensor_tool，需要通过 adb 将 high_freq_sensor_tool push 到平台

```
C:\Windows\System32>adb root

C:\Windows\System32>adb remount
W DM_DEV_STATUS failed for scratch: No such device or address
E [liblpl]No device named scratch
[liblpl]Partition scratch will resize from 0 bytes to 967090176 bytes
[liblpl]Updated logical partition table at slot 0 on device /dev/block/by-name/su
per
[libfs_mgr]Created logical partition scratch on device /dev/block/dm-3
[libfs_mgr]__mount(source=/dev/block/dm-3,target=/mnt/scratch,type=f2fs)=0: Succ
ess
Using overlayfs for /system
Using overlayfs for /vendor
Using overlayfs for /product
[libfs_mgr]__mount(source=overlay,target=/system,type=overlay,upperdir=/mnt/skra
tch/overlay/system/upper)=0
[libfs_mgr]__mount(source=overlay,target=/vendor,type=overlay,upperdir=/mnt/skra
tch/overlay/vendor/upper)=0
[libfs_mgr]__mount(source=overlay,target=/product,type=overlay,upperdir=/mnt/skr
atch/overlay/product/upper)=0
remount succeeded

C:\Windows\System32>adb push d:\high_freq_sensor_tool vendor
d:\high_freq_sensor_tool: 1 file pushed, 1.1 MB/s (69064 bytes in 0.061s)

C:\Windows\System32>adb shell
k6885v1_64:/ # chmod 777 vendor/high_freq_sensor_tool
k6885v1_64:/ #
```

2. high_freq_sensor_tool 的使用方法

指令:

```
./high_freq_sensor_tool -c para1, para2, para3, para4
```

para1→sensorType

1: acc 2: mag 3: orientation 4: gyro
5: light 6: baro 7: temperature 8: proximity
9: gravity 10: linear_accelerate

para2→action

0: disable...1: enable...2: flush...3: cali...4: config_cali...5: selftest... 6: rawdata

para3→delay

采样周期频率 (ns) ,

比如填 80000000 对应的 rate= 12800 = 102400000000/80000000 即 12.8hz

para4→latency

一般填 0.

例如:

```
C:\Windows\System32>adb shell
k6885v1_64:/ # ./vendor/high_freq_sensor_tool -c 1,1,20000000,0
mainThread (4942,4942)
command: [1,1,20000000,0]
looperThread (4942,4943)
data: [1,0,281077745092,-0.222000,-0.093000,9.651000,3]
data: [1,0,281096915977,-0.227000,-0.095000,9.677000,3]
data: [1,0,281116086015,-0.222000,-0.098000,9.675000,3]
data: [1,0,281135260439,-0.215000,-0.093000,9.668000,3]
data: [1,0,281154431846,-0.213000,-0.090000,9.682000,3]
data: [1,0,281173600439,-0.215000,-0.090000,9.668000,3]
data: [1,0,281192769048,-0.217000,-0.090000,9.668000,3]
```

6.3 基于 high_freq_sensor_tool 做 Sensor 校准

6.3.1 ACC 校准方法

1. 在 window 终端, 使用 high_freq_sensor_tool enable acc (para2 = 1)

```
C:\Windows\System32>adb shell
k6885v1_64:/ # ./vendor/high_freq_sensor_tool -c 1,1,20000000,0
mainThread <4942,4942>
command: [1,1,20000000,0]
looperThread <4942,4943>
data: [1,0,281077745092,-0.222000,-0.093000,9.651000,3]
data: [1,0,281096915977,-0.227000,-0.095000,9.677000,3]
data: [1,0,281116086015,-0.222000,-0.098000,9.675000,3]
data: [1,0,281135260439,-0.215000,-0.093000,9.668000,3]
data: [1,0,281154431846,-0.213000,-0.090000,9.682000,3]
data: [1,0,281173600439,-0.215000,-0.090000,9.668000,3]
data: [1,0,281192769048,-0.217000,-0.090000,9.668000,3]
```

2. 在 Windows 再打开一个终端，使用 high_freq_sensor_tool 校准 (para2 = 3)

```
C:\Windows\System32>adb shell
k6885v1_64:/ # ./vendor/high_freq_sensor_tool -c 1,3,0,0
mainThread <4953,4953>
command: [1,3,0,0]
looperThread <4953,4954>
data: [1,3,4495353781,-213.000000,-85.000000,-123.000000,0]
^Csignal_handler0=2 <4953,4953>
signal_handler1=3 <4953,4954>
mainThread exit
./vendor/high_freq_sensor_tool -c 1,3,0,0
mainThread <4955,4955>
command: [1,3,0,0]
looperThread <4955,4956>
data: [1,3,4495353654,-213.000000,-85.000000,-123.000000,1]
^Csignal_handler0=2 <4955,4955>
signal_handler1=3 <4955,4956>
mainThread exit
k6885v1_64:/ #
```

3. 检查校准结果

校准成功后，校准结果会保存在 JSON 文件：

```
k79v1_64:/ # cat /mnt/vendor/nvcfg/sensor/acc_cali.json
{
  "acc_cali": [
    -12,
    201,
    -45
  ]
}
```

6.3.2 GYRO 校准方法 (与 ACC 相同)

6.3.3 PS 校准方法

1. 在 window 终端，使用 high_freq_sensor_tool 直接校准 (para2 = 3)

```
k6885v1_64:/ # ./vendor/high_freq_sensor_tool -c 8,3,0,0
mainThread (5840,5840)
command: [8,3,0,0]
looperThread (5840,5841)
psCaliThread (5840,5842)
Please put phone on a horizontal plane.
NOTE: Donot cover P-Sensor.
Press 'y' to start calibration or 'n' to exit:
```

2. type "y" 计算最大值与最小值

```
Press 'y' to start calibration or 'n' to exit: y
P-Sensor is sampling min value now, please wait...
P-Sensor get min value success:0
Please cover the P-Sensor.
Press 'y' to start calibration or 'n' to exit:
```

```
Press 'y' to start calibration or 'n' to exit: y
P-Sensor is sampling max value now, please wait...
P-Sensor calculate max value success:268
Press 'y' to start P-Sensor calibration or 'n' to exit:
```

```
Press 'y' to start calibration or 'n' to exit: y
P-Sensor is sampling max value now, please wait...
P-Sensor calculate max value success:268
Press 'y' to start P-Sensor calibration or 'n' to exit:
```

```
Press 'y' to start P-Sensor calibration or 'n' to exit: y
P-Sensor is doing calibration now, please wait...
ps_min_largest:0, ps_min_smallest:0
calculate unit:<0>
P-Sensor threshold:89, 89
P-Sensor calibration success
```

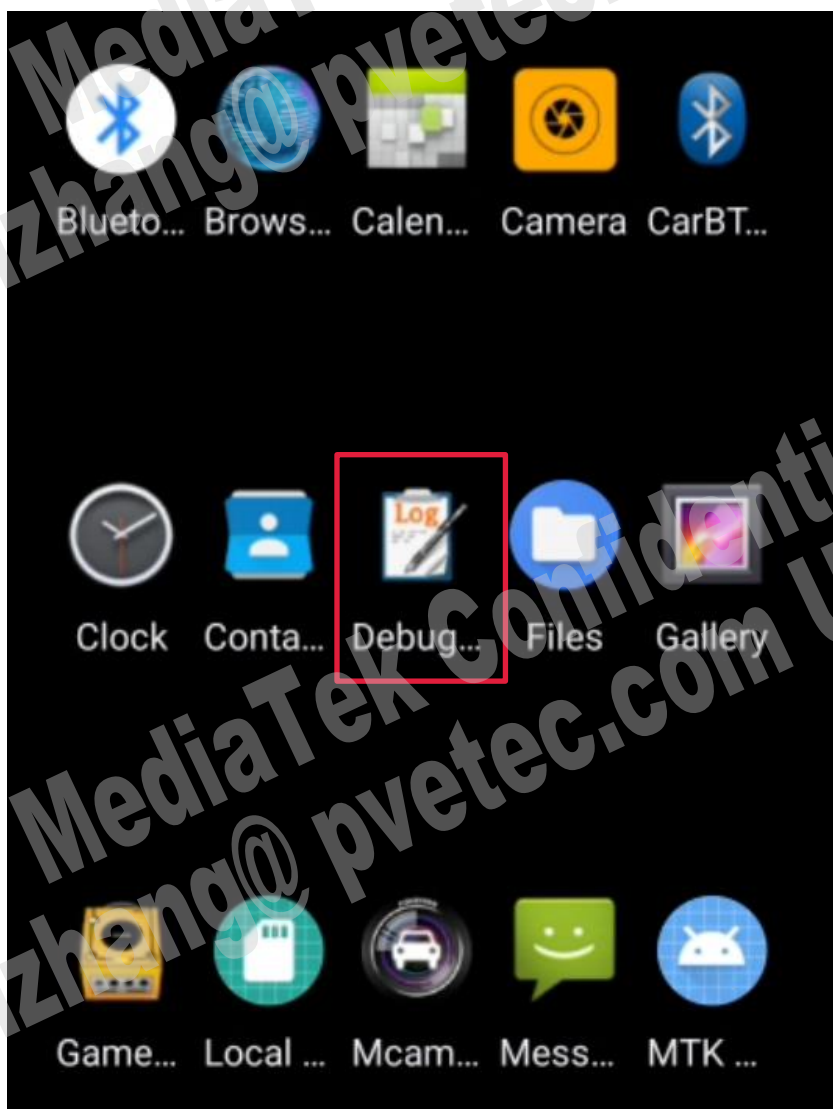
3. 检查校准结果

```
k79v1_64:/ # cat /mnt/vendor/nucfg/sensor/ps_cali.json
{
  "ps_cali": [
    89,
    89
  ]
}
>k79v1_64:/ #
```

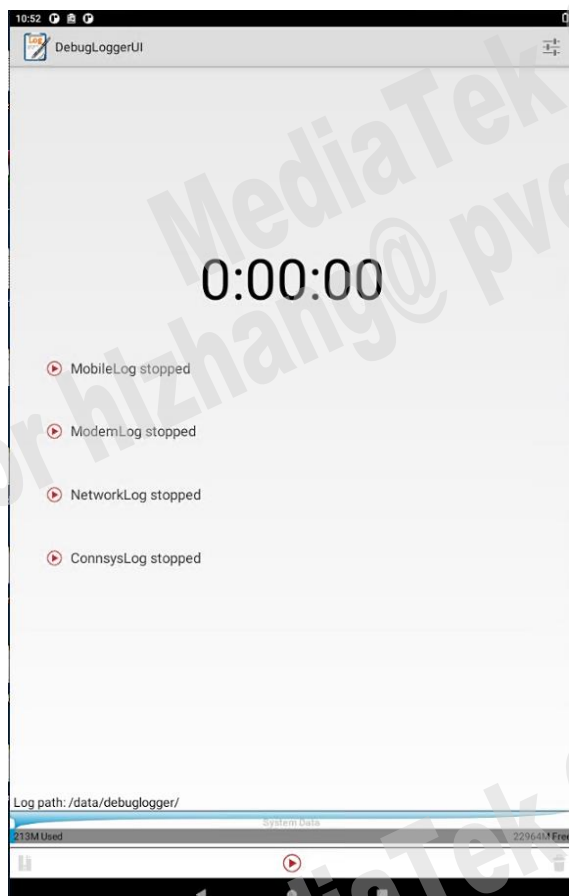
7 如何抓取 Sensor Log

如何导出 Sensor 相关的 log

1. 在屏幕打开 DebugLoggerUI

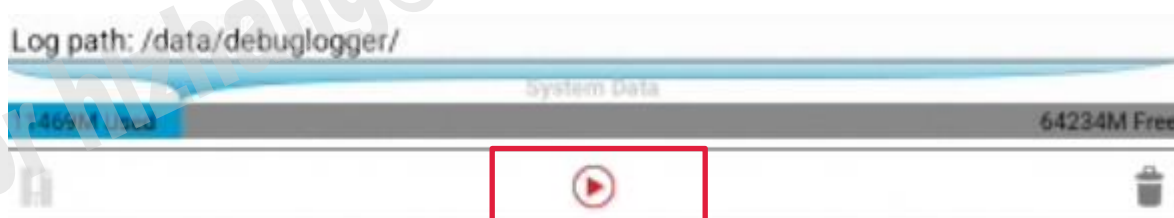


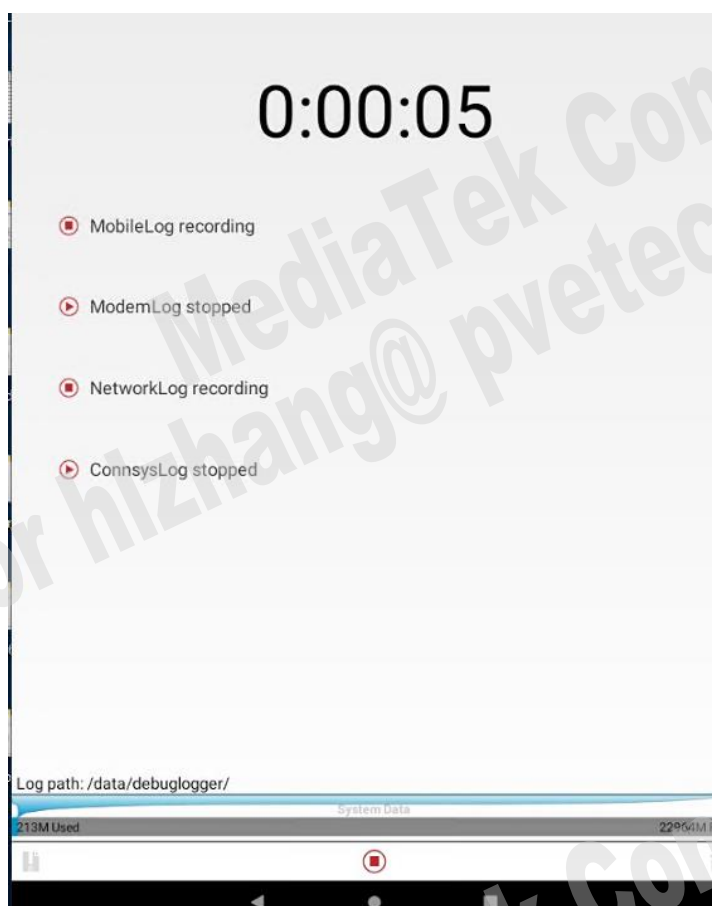
打开后界面如下：



2. 点击启动按钮

启动后，软件会将缓冲区的 log 保存到 txt 文件（包括开机启动的 log）





3. log 的导出

为了确保存储足够的 log，建议保存 40s 以上的 log，再点击停止按钮

导出 log 的指令：

```
adb pull /data/debuglogger
```

4. 关键 log 信息：

查看 SCP 的启动 log，检查各个 Sensor 是否有成功 probe 起来：

```
[3.535](0)[0] principal.smg230 get chip id 0xf
[3.536](0)[0] principal.smg230 get chip id 0xf
[3.536](0)[0] principal.smg230 smg230 setup
[3.567](0)[0] principal.smg230 smg230 setup done
[3.568](0)[0] principal.smi230 DRIVER VERSION:0x1
[3.589](0)[0] principal.smi230 chip id 0x1f
[3.589](0)[0] principal.smi230 get chip id 0x1f
[3.594](0)[0] principal.smi230 read 0x53:0xa 0x54:0xa ret 0
[3.596](0)[0] principal.smi230 read 0x7c:0x0 0x7d:0x0 ret 0
[3.597](0)[0] principal.smi230 begin load config file 6144
[3.598](0)[0] principal.transceiver task 0 id 2
```

如图中 smi230driver 的启动 log.

设置采样频率，上报频率等：

```
[336.755](0)[0] deputy.motion get magnitude: 0.205555
[336.816](0)[0] principal.transceiver enable: 1 20000000 0
```

如图中，为 enable 加速度计，采样间隔为 20000000ns，上报间隔为 0.

8 案例分析

8.1 ALPS08300249 平板 8 字校准后，地磁数据与机械指南针对比相差大于 5

本体客户反馈部分机器磁感即使经过校准，指南针也会有 90 度的方向偏差。

90 度方向偏差首先要考虑改动 SCP 配置的 direction。在驱动中会读取 direction 并将实际的数据进行 remap。

若无特殊情况，贴在 PCB（对于屏幕方向）正面的器件其方向应当为 0 1 2 3 其中的一个，贴在背面的应为 4 5 6 7 其中的一个。可以查看源码确认 direction 对 remap 的影响。

本题实际上属于例外情况。客户产品特殊，同一份 load 支持了多种 PCB，因此 bring up 时就联合 vendor 开发了客制化特性，在 Kernel 侧读取 board ID，并下发特殊指令重置底层的 direction。

由于 OEM 换人，升级 Kernel 版本时客户方信息不同步导致了这个问题。

在正常排查未发现问题时，需要考虑是否存在客制化因素。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿赔偿责任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。